

Security Audit

Lizard Labs (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Audit Findings	14
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lizard Labs team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

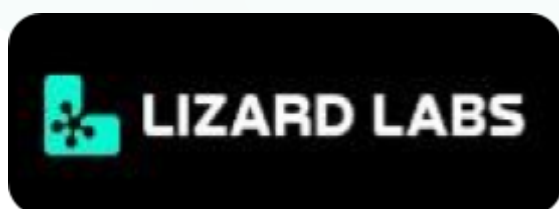
Lizard Labs specializes in Web3 innovation, focusing on decentralized technologies and community-driven projects. They provide tools for streamlined project development and blockchain integration. Their services and solutions cater to businesses seeking to leverage blockchain for transparency, security, and user empowerment. To build engagement, they emphasize community-focused strategies, reward mechanisms, and scalable blockchain solutions. By merging Web3 tech with marketing strategies like token-based loyalty programs and NFT-based brand experiences, Lizard Labs enables brands to foster deeper connections with digital-native audiences.

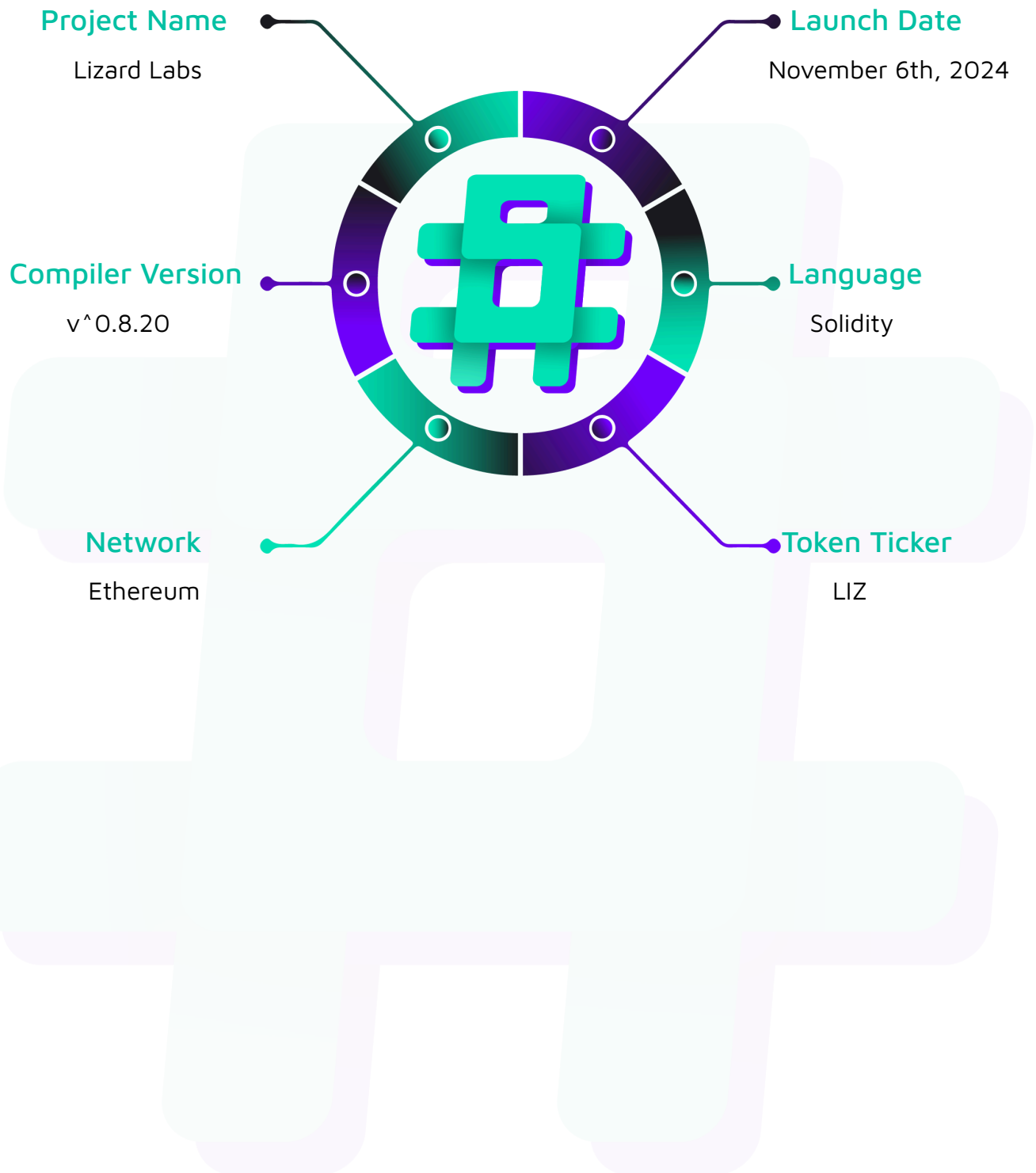
Project Name: Lizard Labs

Compiler Version: ^0.8.20

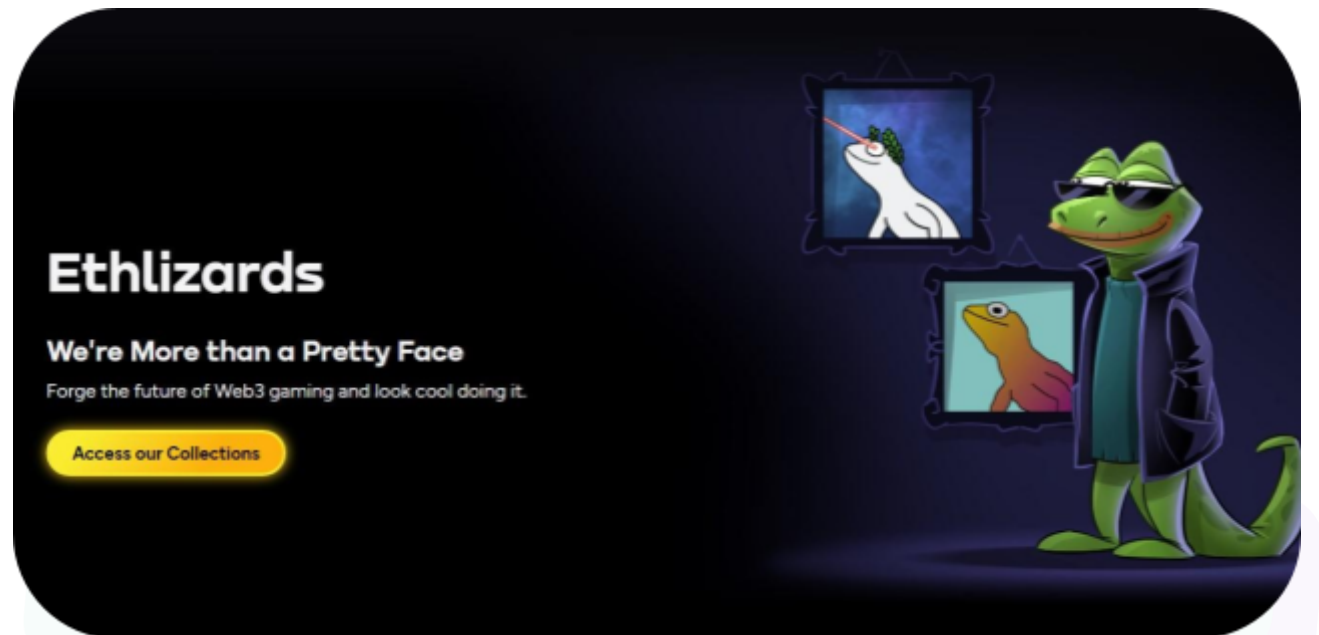
Website: www.lizlabs.io

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Lizard Labs project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Lizard Labs Smart Contracts
Platform	Ethereum / Solidity
Audit Date	October, 2024
Contract 1	MergedEmissionScheduleV1.sol
Contract 1 MD5 Hash	2e06804e61a23b139c4bd780835476c8
Contract 2	RevDisScheduleV1.sol
Contract 2 MD5 Hash	01924cac6c88dcd07aab9905f92e6de4
Contract 3	UniformEpochEmissionScheduleV1.sol
Contract 3 MD5 Hash	06f0eab9924622cc9b54744def036dd0
Contract 4	MulticallHelper.sol
Contract 4 MD5 Hash	024f54d541e14bcadcbdaab9186b20bc
Contract 5	SlashedStakingV1.sol
Contract 5 MD5 Hash	65156bb882e05ce11237fb7db00e39ec
Contract 6	TimeWeightedStakingV1.sol
Contract 6 MD5 Hash	e52a14e6ee02037905928e52f3ad019c
Contract 7	ScheduledTokenDistributorV1.sol
Contract 7 MD5 Hash	f0cf501b66decc364f1bb61fd0e58f1f
Contract 8	SimpleTokenDistributorV1.sol
Contract 8 MD5 Hash	c6c9c7fad98ca53eb276259ea0870a1c
Contract 9	LizVestingV1.sol

Contract 9 MD5 Hash	4fe130eefdeea649294037a11bfa4e5f
Contract 10	LizcoinERC20.sol
Contract 10 MD5 Hash	b1d3af1b5f8b0bc8622165745ecde432

Description	Mainnet Deployed Addresses
LIZ ERC20/1363 Token	0xAF4144cd943ed5362Fed2BaE6573184659CBe6FF
cLIZ ERC20 Token	0x0F9dc0c0A46733c8b9a6C2E4850913Ed31d31205
vLIZ ERC20 Token	0xe20C4edb8440CaDD4001c144B4F38576d1AA3820
veLIZ ERC20 Token	0xC817COB518e8Fc98034ad867d679d4f8A284BFBE
Liz Vesting	0x895ecdCAC6431272946Ec615eD368d2f42fC2b44
Slashed Staking	0xEf2E841AA9F49cc0E54697A4Afa6361eA24d682F
Scheduled Distributor	0x363edec51cE53F6965296A0829411D71EDa7845B
Simple Distributor 1	0x5f5C05Fa373529bC1B235780c97622cb77b35d37
Simple Distributor 2	0x272775f8EeCAB6c9B951BfDbf6dCb008544c9702
Emission Schedule 1	0x23aeb1F6ebbea76a1e68a7520f8b580bE1ca66b7
Emission Schedule 2	0x190b361c5CcA8E8c9f7faaE7eF69605b455903E2
Emission Schedule 3	0x08c2415e130d3849D9F40b57e71c1e8dD228209B
Multicall Helper	0xA16C965b4e6c6B6D67722bD753eb23391e1C7565
Uniswap V2 Factory	0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f
Uniswap V2 Router V2	0x7a250d5630B4cF539739dF2C5dAcB4c659F2488D
WETH9	0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2
LIZ/ETH UniswapV2 LP	0xD47B93360EAADBA2678c30F64209a42b9800cEE4

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Lazy-Sol contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All findings uncovered during automated and manual analysis were meticulously reviewed and applicable findings are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 QAs

1 Gas Optimisation

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MergedEmissionScheduleV1.sol Combines two emission schedules into one	Contract achieves this functionality.
RevDisScheduleV1.sol Revenue distribution functionality delivered in a form of the emission schedule, making it possible to inject into staking contract either directly or in a merged form	Contract achieves this functionality.
UniformEpochEmissionScheduleV1.sol Emission schedule which breaks the emission period into the epochs of the equal duration and has its emission rates defined for every epoch	Contract achieves this functionality.
MulticallHelper.sol Multicall support with a function to batch together multiple calls in a single external call	Contract achieves this functionality.
SlashedStakingV1.sol Fixed-Term Time Weighted staking extension supporting the reward capping/slashing mechanism	Contract achieves this functionality.
TimeWeightedStakingV1.sol Fixed lock-up period staking, based on Synthetix reward pool calculation mechanics	Contract achieves this functionality.
ScheduledTokenDistributorV1.sol Designed to get the tokens from the EmissionSchedule instance, which guarantees no tokens are released above the scheduled	Contract achieves this functionality.

amounts	
SimpleTokenDistributorV1.sol Get the tokens from the treasury wallet specified	Contract achieves this functionality.
LizVestingV1.sol ERC20 token vesting contract for Lizcoin ERC20 token vesting featuring	Contract achieves this functionality.
LizcoinERC20.sol Governance token for the Gaming Sub-DAO of Lizard Labs	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Lizard Labs project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Lizard Labs project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

QA

[Q-01] `LizVestingV1#postConstruct` - `365 * 24 * 3600` could be replaced with `365 days` for readability

Description

The `postConstruct` function uses `365 * 24 * 3600` while some variable assignments. `365 * 24 * 3600` could be replaced with `365 days` to improve code readability.

Recommendation

Replace `365 * 24 * 3600` with `365 days`.

Status

Resolved

[Q-02] `Contracts` - Floating pragma

Description

The contracts use `pragma solidity ^0.8.4`.

This can allow the contracts to be deployed with the wrong pragma version which is different from the one the contracts were tested with.

Recommendation

Lock the pragma version.

Status

Acknowledged

[Q-03] TimeWeightedStakingV1#__updateState - `TODO` comment could be removed

Description

The `__updateState` function still has a `TODO` comment.

Having such comments on the production would make the code less readable and reliable.

Recommendation

Remove the `TODO` comment.

Status

Resolved

Gas

[G-01] MerkleTokenDistributor - ClaimRequest struct could be packed

Description

The `ClaimRequest` struct could be packed by moving `validAfter` and `validBefore` elements right after `withCallback` element.

Recommendation

Move `validAfter` and `validBefore` elements right after `withCallback` element.

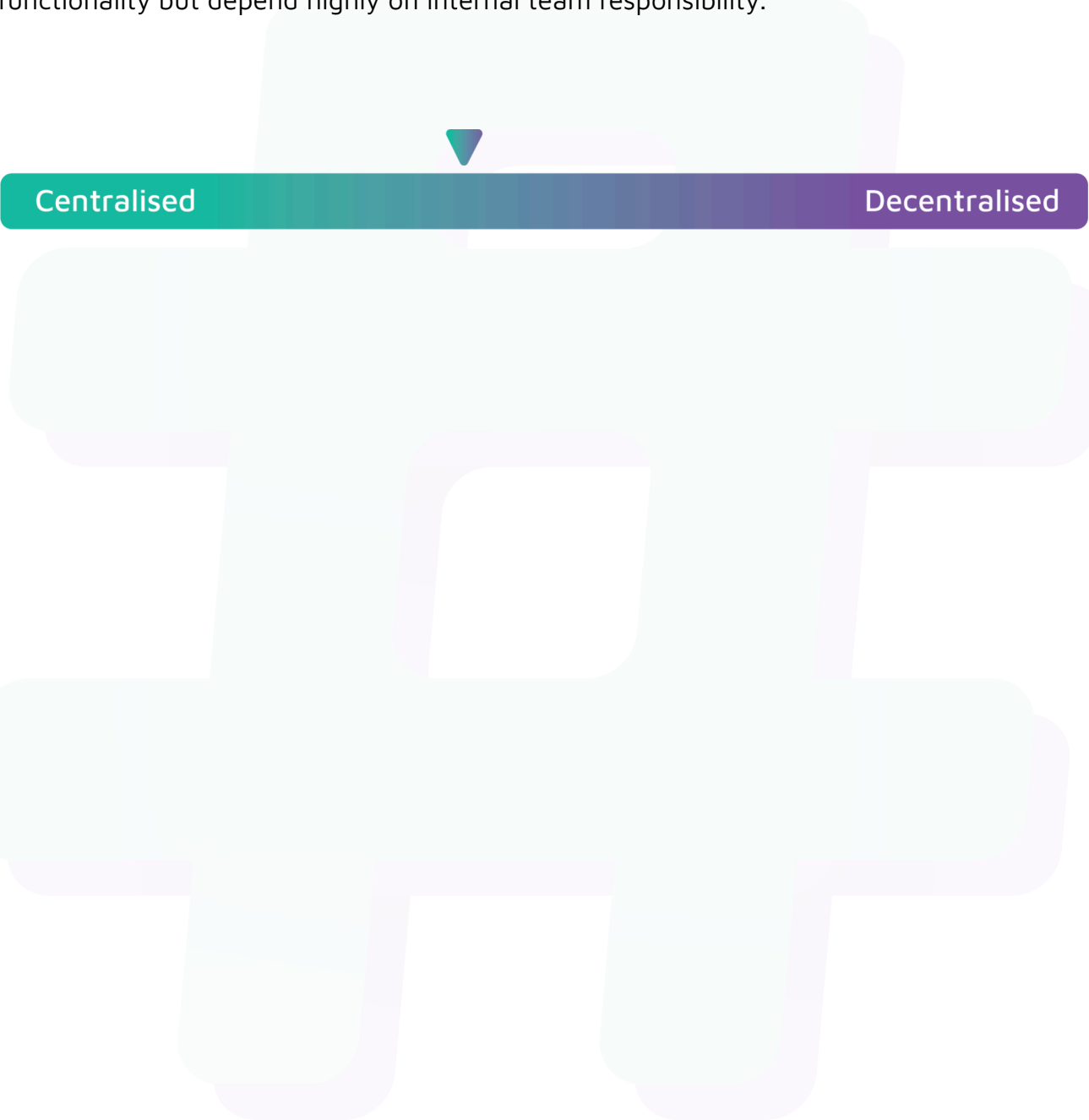
Status

Acknowledged

Centralisation

The Lizard Labs project values security and utility over decentralisation.

The role based executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Lizard Labs project seems to have a sound and well-tested code base, now that our findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.